

```
In [90]: # Ezinne Akabueze, CMOR 220, Spring 2026, Project 7: Newton's Method w
# Project7-Newton's Method.ipynb
# This coding program is tasked to
# Last modified: 4/07/26
```

```
In [91]: import numpy as np
import matplotlib.pyplot as plt

# Defining all necessary function
def myownpolyder(q): # calculates the derivative of a polynomial
    # Input: q is the different coefficients of the polynomial, where
    degree = len(q) - 1 # etsablshes the degree of the polynomial
    dq = [] # starts an empty list for when we have the derivatives
    for i in range(len(q)-1):
        dq.append(q[i] * (degree-i)) # calculates dervivative then add
    return dq # Output: the derivative of the polynomial

def qnewt (q, xt, yt, maxiter): # Newton's method, produces a plot of
    # Inputs: q is the coefficients, xt is the guess for the root, yt
    x = np.arange(xt[0], xt[1] + xt[2], xt[1]) # sets the range of wha
    y = np.arange(yt[0], yt[1] + yt[2], yt[1]) # sets the range of wha
    X, Y = np.meshgrid(x, y)
    Z = X + 1j * Y
    roots = np.roots(q) # uses matlab function to find the equation's

    for i in range(maxiter):
        Z = Z - (np.polyval(q,Z))/(np.polyval(myownpolyder(q),Z)) # fi

    plt.figure()
    color = ['red', 'yellow', 'green', 'blue'] # sets the colors for t
    for i in range(len(roots)):
        rows, columns = np.where(np.abs(Z - roots[i]) < .001) # estabi
        plt.plot(x[columns], y[rows], '.', color = color[i], markersiz
    plt.title(figure_title(q)) # sets the title of the plot to be the
    plt.xlabel('x = real values')
    plt.ylabel('y = imaginary values')
    plt.show()
    # Output: the root of the polynomial

def figure_title(q): # generates the plot title
    # Input: q is the coefficients of the polynomial
    terms = []
    degree = len(q) - 1
    for i in range(len(q)):
        power = degree - i
        coefficient = q[i]
        if abs(coefficient) == 0:
            continue
        if coefficient != 0:
```

```

        if power > 1:
            term = f"{coefficient}z^{power}"
        elif power == 1:
            term = f"{coefficient}z"
        else:
            term = f"{coefficient}"
        terms.append(term)
    full_str = " + ".join(terms)
    full_str = full_str.replace(' 1z', 'z')
    full_str = full_str.replace('+ -', '- ')

    return full_str
# Output: the title of the plot, aka polynomial itself

```

In []:

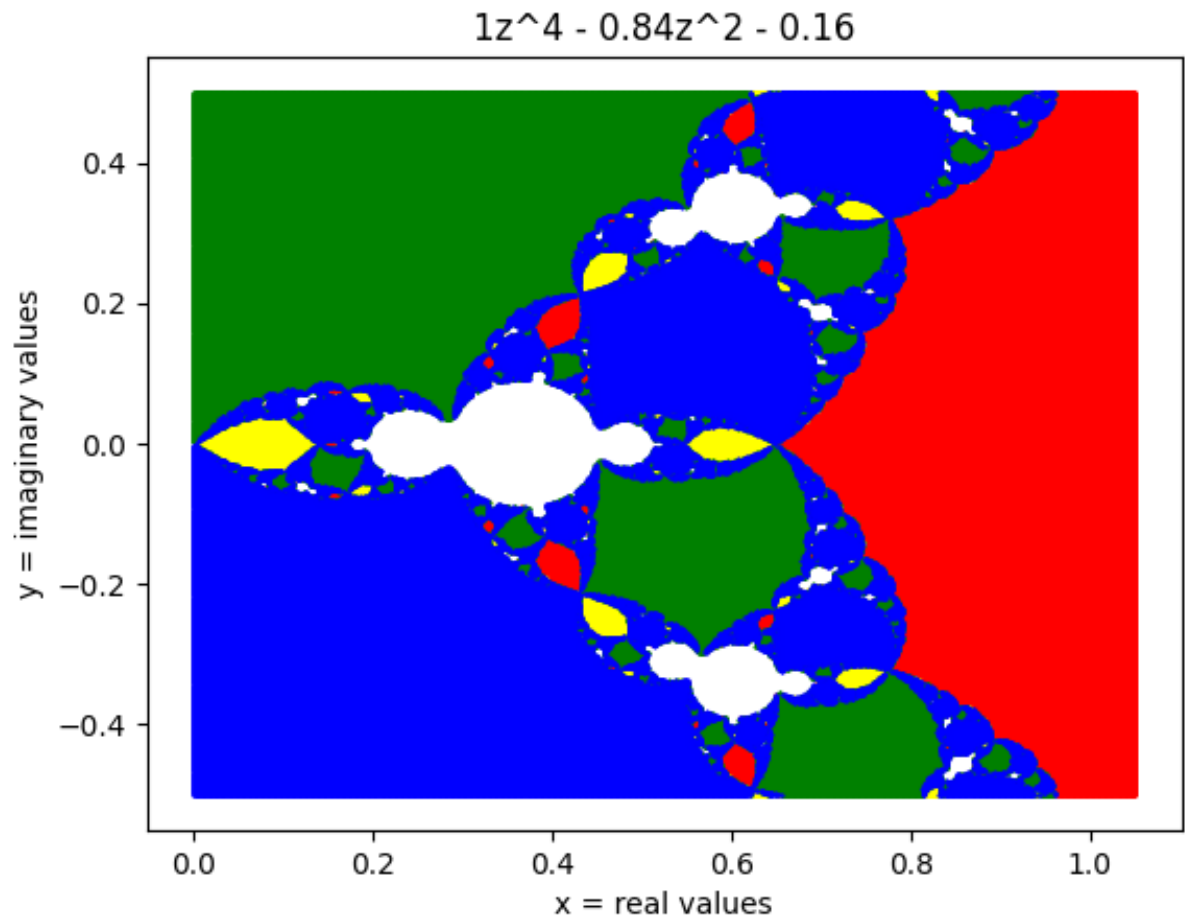
```

In [92]: # Figure 1: Sampling of the Newton basins for the equation
print("part 1")
# Parameters from Project Description [cite: 333, 812, 813]
xt_param = [0, 0.001, 1.05]
yt_param = [-0.5, 0.001, 0.5]
it_limit = 20

# Polynomial 1 [cite: 329, 448]
q1 = [1, 0, -0.84, 0, -0.16]
qnewt(q1, xt_param, yt_param, it_limit)

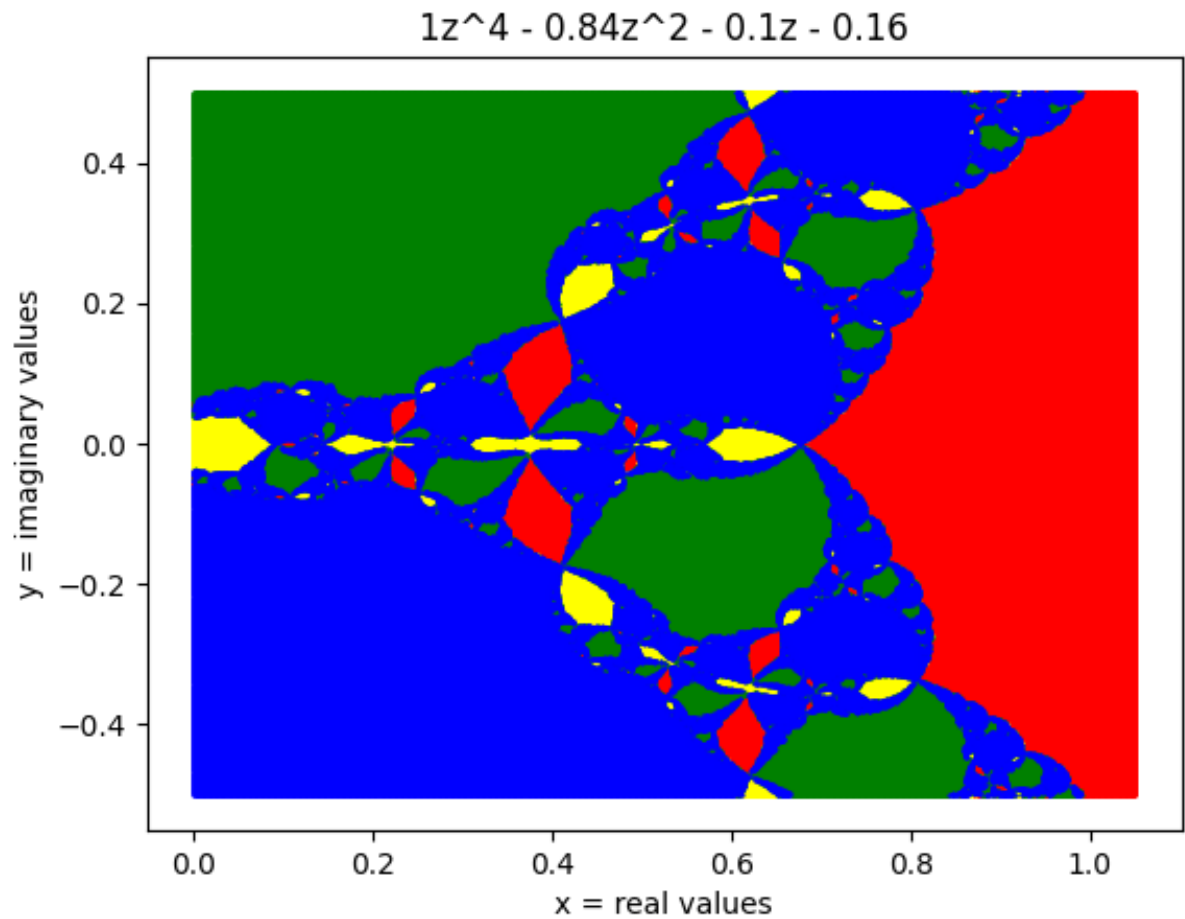
```

part 1



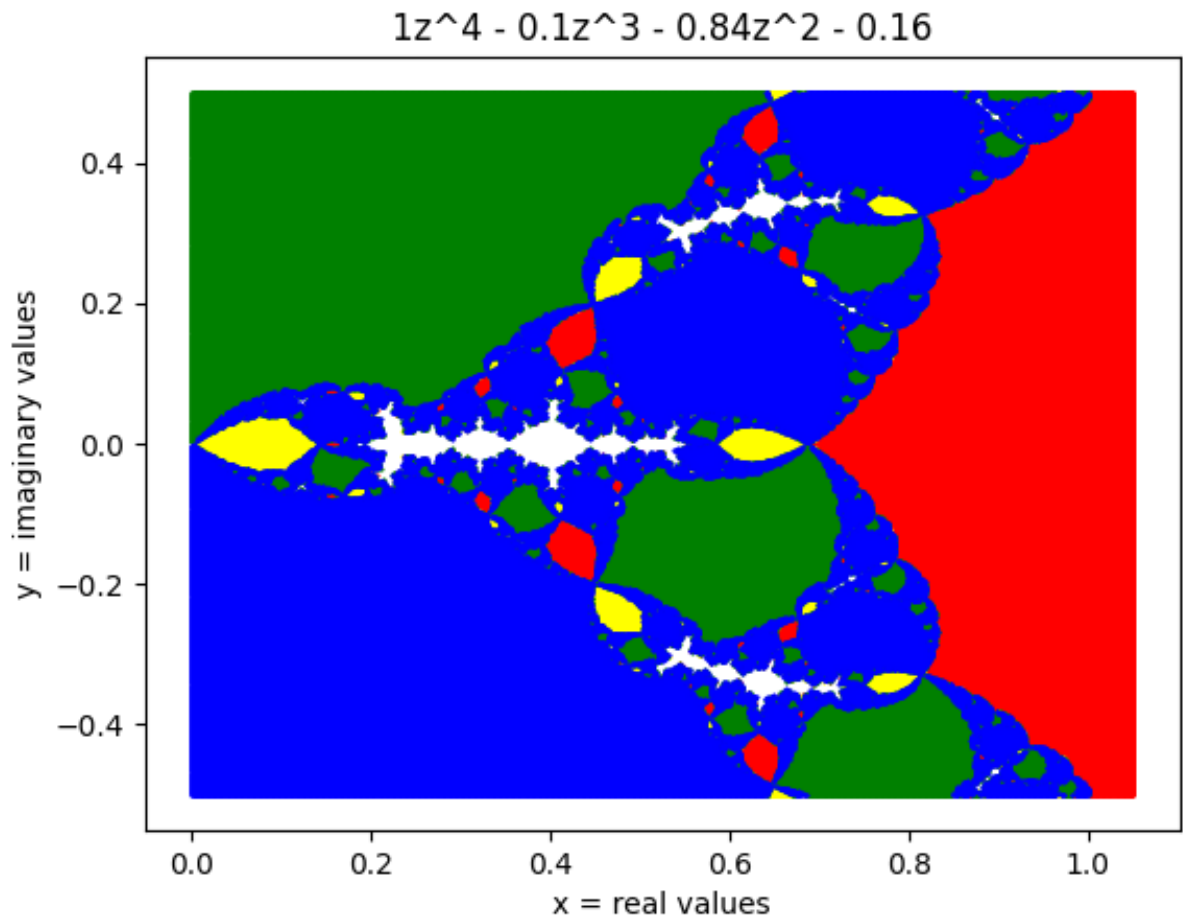
```
In [93]: # Figure 2: The Newtons basins for the equation
print("part 2")
# Polynomial 2 [cite: 330, 452, 814]
q2 = [1, 0, -0.84, -0.1, -0.16]
qnewt(q2, xt_param, yt_param, it_limit)
```

part 2



```
In [94]: print("part 3")  
# Polynomial 3 [cite: 331, 453, 816]  
q3 = [1, -0.1, -0.84, 0, -0.16]  
qnewt(q3, xt_param, yt_param, it_limit)
```

part 3



```
In [95]: print("part 4")  
# Polynomial 4 [cite: 332, 454, 819]  
q4 = [1, -0.1j, -0.84, 0, -0.16]  
qnewt(q4, xt_param, yt_param, it_limit)
```

part 4

$$1z^4 + (-0-0.1j)z^3 - 0.84z^2 - 0.16$$

